WetInk Next

Полный исходный код P0 - P5

49 файлов: Kotlin, тесты, Android, Gradle и CI

Включая P5: round opaque brush, stroke target и premultiplied preview/commit

.github/workflows/android.yml

```yaml
name: Android CI

on:
  push:
  pull_request:

jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '17'
      - uses: android-actions/setup-android@v3
      - uses: gradle/actions/setup-gradle@v4
      - run: ./gradlew :app:compileDebugKotlin :app:testDebugUnitTest :app:lintDebug
```

.gitignore

```
.gradle/
build/
local.properties
*.iml
.idea/
**/build/
```

settings.gradle.kts

```
pluginManagement {
    repositories {
        google()
        mavenCentral()
        gradlePluginPortal()
    }
}

dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}

rootProject.name = "WetInk Next"
include(":app")
```

build.gradle.kts

```
plugins {
    id("com.android.application") version "8.6.1" apply false
    id("org.jetbrains.kotlin.android") version "2.0.21" apply false
    id("org.jetbrains.kotlin.plugin.compose") version "2.0.21" apply false
}
```

gradle.properties

```
org.gradle.jvmargs=-Xmx2048m -Dfile.encoding=UTF-8
android.useAndroidX=true
android.overridePathCheck=true
kotlin.code.style=official
org.gradle.daemon=true
org.gradle.caching=true
```

gradle/wrapper/gradle-wrapper.properties

```
distributionBase=GRADLE_USER_HOME
distributionPath=wrapper/dists
distributionUrl=https\://services.gradle.org/distributions/gradle-8.7-bin.zip
networkTimeout=10000
validateDistributionUrl=true
zipStoreBase=GRADLE_USER_HOME
zipStorePath=wrapper/dists
```

gradlew

```
#!/bin/sh

APP_HOME=$(CDPATH= cd -- "$(dirname -- "$0")" && pwd -P) || exit 1
JAVA_CMD="${JAVA_HOME:+$JAVA_HOME/bin/}java"
exec "$JAVA_CMD" -classpath "$APP_HOME/gradle/wrapper/gradle-wrapper.jar" org.gradle.wrapper.GradleWrapperMain "$@"
```

gradlew.bat

```
@rem Minimal Gradle startup script for Windows.
@echo off
setlocal
set APP_HOME=%~dp0
if defined JAVA_HOME goto javaHome
java -classpath "%APP_HOME%gradle\wrapper\gradle-wrapper.jar" org.gradle.wrapper.GradleWrapperMain %*
goto end
:javaHome
"%JAVA_HOME%\bin\java.exe" -classpath "%APP_HOME%gradle\wrapper\gradle-wrapper.jar" org.gradle.wrapper.GradleWrapperMain %*
:end
```

app/build.gradle.kts

```
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
    id("org.jetbrains.kotlin.plugin.compose")
}

android {
    namespace = "com.wetinknext"
    compileSdk = 36

    defaultConfig {
        applicationId = "com.wetinknext"
        minSdk = 26
        targetSdk = 36
        versionCode = 1
        versionName = "0.1.0"
        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            isMinifyEnabled = false
        }
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }

    kotlinOptions {
        jvmTarget = "17"
    }

    buildFeatures {
        compose = true
    }

}

dependencies {
    val composeBom = platform("androidx.compose:compose-bom:2024.09.03")
    implementation(composeBom)
    androidTestImplementation(composeBom)

    implementation("androidx.core:core-ktx:1.13.1")
    implementation("androidx.activity:activity-compose:1.9.2")
    implementation("androidx.compose.ui:ui")
    implementation("androidx.compose.ui:ui-tooling-preview")
    implementation("androidx.compose.material3:material3")

    testImplementation("junit:junit:4.13.2")

    debugImplementation("androidx.compose.ui:ui-tooling")
    debugImplementation("androidx.compose.ui:ui-test-manifest")
}
```

app/src/main/AndroidManifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <application
        android:allowBackup="true"
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:theme="@style/Theme.WetInkNext">
        <activity
            android:name=".app.MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

app/src/main/res/values/strings.xml

```
<resources>
    <string name="app_name">WetInk Next</string>
</resources>
```

app/src/main/res/values/themes.xml

```
<resources>
    <style name="Theme.WetInkNext" parent="android:style/Theme.Material.NoActionBar" />
</resources>
```

```
package com.wetinknext.app

import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.compose.ui.viewinterop.AndroidView
import com.wetinknext.engine.core.PaintSurfaceView
import com.wetinknext.ui.theme.WetInkNextPalette

@Composable
fun EditorScreen() {
    val palette = WetInkNextPalette.default

    Box(
        modifier = Modifier
            .fillMaxSize()
            .background(palette.appBg),
    ) {
        AndroidView(
            modifier = Modifier.fillMaxSize(),
            factory = { context -> PaintSurfaceView(context) },
        )

        Row(
            modifier = Modifier
                .align(Alignment.TopEnd)
                .padding(top = 12.dp, end = 12.dp)
                .background(palette.panelBg.copy(alpha = 0.94f), RoundedCornerShape(24.dp))
                .border(1.dp, palette.panelStroke, RoundedCornerShape(24.dp))
                .padding(horizontal = 10.dp, vertical = 8.dp),
            horizontalArrangement = Arrangement.spacedBy(10.dp),
            verticalAlignment = Alignment.CenterVertically,
        ) {
            repeat(7) {
                Box(Modifier.size(18.dp).background(palette.textPrimary.copy(alpha = 0.9f), CircleShape))
            }
            Box(Modifier.size(26.dp).background(palette.accent, CircleShape))
        }

        Column(
            modifier = Modifier
                .align(Alignment.CenterStart)
                .padding(start = 12.dp)
                .background(palette.panelBg.copy(alpha = 0.94f), RoundedCornerShape(24.dp))
                .border(1.dp, palette.panelStroke, RoundedCornerShape(24.dp))
                .padding(horizontal = 10.dp, vertical = 12.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
        ) {
            repeat(5) {
                Box(Modifier.size(22.dp).background(palette.textPrimary.copy(alpha = 0.9f), CircleShape))
            }
        }

        Column(
            modifier = Modifier
                .align(Alignment.BottomCenter)
                .padding(bottom = 20.dp)
                .background(palette.panelBg.copy(alpha = 0.94f), RoundedCornerShape(28.dp))
                .border(1.dp, palette.panelStroke, RoundedCornerShape(28.dp))
                .padding(horizontal = 18.dp, vertical = 12.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
        ) {
            Text("WetInk Next", color = palette.textPrimary, fontSize = 16.sp)
            Text("P0: app boots, surface attached", color = palette.textSecondary, fontSize = 12.sp)
        }
    }
}
```

app/src/main/java/com/wetinknext/app/MainActivity.kt

```
package com.wetinknext.app

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent { EditorScreen() }
    }
}
```

```
package com.wetinknext.engine.brush

enum class BrushRenderMode { STAMP, RIBBON, WET }
enum class BlendPolicy { NORMAL_BUILDUP, NON_BUILDUP }
data class RibbonSettings(val minWidthRatio: Float = 0.06f, val taperStartPx: Float = 8f, val taperEndPx: Float = 14f, val miterLimit: Float = 3f, val aaWidthPx: Float = 1f)
data class WetSettings(val wetness: Float = 0f, val spread: Float = 0f, val bleed: Float = 0f)
data class BrushSettings(
    val name: String = "Debug Stamp", val category: String = "Debug", val renderMode: BrushRenderMode = BrushRenderMode.STAMP,
    val baseRadiusPx: Float = 14f, val opacity: Float = 1f, val flow: Float = 1f, val spacing: Float = 0.16f,
    val spacingUsesDiameter: Boolean = true, val hardness: Float = 1f, val colorArgb: Long = 0xFF000000L,
    val smoothing: Float = 0f, val streamline: Float = 0f, val antiAliasLevel: Int = 2, val useTempStrokeBuffer: Boolean = false,
    val pressureToSize: Boolean = true, val pressureToOpacity: Boolean = true, val pressureGamma: Float = 1f,
    val minSizeRatio: Float = 0.05f, val velocityToSize: Float = 0f, val velocityToOpacity: Float = 0f,
    val tiltToSize: Float = 0f, val tiltToOpacity: Float = 0f, val tiltToRotation: Float = 0f,
    val rotationJitter: Float = 0f, val sizeJitter: Float = 0f, val opacityJitter: Float = 0f, val scatter: Float = 0f,
    val shapeAssetPath: String? = null, val grainAssetPath: String? = null, val grainScale: Float = 1f,
    val grainCanvasLocked: Boolean = true, val rgbToAlpha: Boolean = false, val textureContrast: Float = 1f,
    val textureDepth: Float = 1f, val pixelPen: Boolean = false, val squareStroke: Boolean = false,
    val noAntialias: Boolean = false, val blendPolicy: BlendPolicy = BlendPolicy.NORMAL_BUILDUP,
    val ribbon: RibbonSettings = RibbonSettings(), val wet: WetSettings = WetSettings(),
)
```

```
package com.wetinknext.engine.brush

import kotlin.math.pow

object ColorSpaces {
    fun srgbToLinear(value: Float): Float { val v=value.coerceIn(0f,1f); return if(v<=.04045f)v/12.92f else ((v+.055f)/1.055f).pow(2.4f) }
    fun linearToSrgb(value: Float): Float { val v=value.coerceIn(0f,1f); return if(v<=.0031308f)v*12.92f else 1.055f*v.pow(1f/2.4f)-.055f }
    fun srgb8ToLinear(argb: Long,out:FloatArray){require(out.size>=3);out[0]=srgbToLinear(((argb shr 16)and 0xFF)/255f);out[1]=srgbToLinear(((argb shr 8)and 0xFF)/255f);out[2]=srgbToLinear((argb and 0xFF)/255f)}
}
```

```
package com.wetinknext.engine.brush

import java.nio.ByteBuffer
import java.nio.ByteOrder

class DabBuffer(val capacity: Int = DEFAULT_CAPACITY) {
    init { require(capacity > 0) }
    val floats = ByteBuffer.allocateDirect(capacity * FLOATS_PER_DAB * Float.SIZE_BYTES).order(ByteOrder.nativeOrder()).asFloatBuffer()
    var count = 0; private set; var overflowCount = 0L; private set
    fun clear() { count = 0; overflowCount = 0L; floats.clear() }
    fun add(x: Float, y: Float, radius: Float, rotation: Float, alpha: Float): Boolean { if (count >= capacity) { overflowCount++; return false }; floats.put(x); floats.put(y); floats.put(radius); floats.put(rotation); floats.put(alpha); count++; return true }
    fun prepareForUpload() { floats.position(0); floats.limit(count * FLOATS_PER_DAB) }
    /** Restores the direct buffer for appending after a GL upload. */
    fun finishUpload() {
        floats.limit(capacity * FLOATS_PER_DAB)
        floats.position(count * FLOATS_PER_DAB)
    }
    companion object { const val FLOATS_PER_DAB = 5; const val DEFAULT_CAPACITY = 8192 }
}
```

```
package com.wetinknext.engine.brush

import android.opengl.GLES30
import com.wetinknext.engine.gl.GlProgram
import com.wetinknext.engine.gl.RenderTarget
import com.wetinknext.engine.gl.ShaderLib
import java.nio.ByteBuffer
import java.nio.ByteOrder

class DabRenderer(private val maxDabs:Int){private var program:GlProgram?=null;private var vao=0;private var quad=0;private var instances=0;private var matrix=-1;private var color=-1
 fun create(){release();program=GlProgram(ShaderLib.dabVertex,ShaderLib.dabFragment);program!!.use();matrix=GLES30.glGetUniformLocation(program!!.id,"uCanvasToClip");color=GLES30.glGetUniformLocation(program!!.id,"uColorLinear");val v=floatArrayOf(-1f,-1f,1f,-1f,-1f,1f,1f,1f);val b=ByteBuffer.allocateDirect(v.size*4).order(ByteOrder.nativeOrder()).asFloatBuffer();b.put(v).position(0);val ids=IntArray(1);GLES30.glGenVertexArrays(1,ids,0);vao=ids[0];GLES30.glGenBuffers(1,ids,0);quad=ids[0];GLES30.glGenBuffers(1,ids,0);instances=ids[0];GLES30.glBindVertexArray(vao);GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER,quad);GLES30.glBufferData(GLES30.GL_ARRAY_BUFFER,v.size*4,b,GLES30.GL_STATIC_DRAW);GLES30.glEnableVertexAttribArray(0);GLES30.glVertexAttribPointer(0,2,GLES30.GL_FLOAT,false,0,0);GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER,instances);GLES30.glBufferData(GLES30.GL_ARRAY_BUFFER,maxDabs*20,null,GLES30.GL_DYNAMIC_DRAW);GLES30.glEnableVertexAttribArray(1);GLES30.glVertexAttribPointer(1,4,GLES30.GL_FLOAT,false,20,0);GLES30.glVertexAttribDivisor(1,1);GLES30.glEnableVertexAttribArray(2);GLES30.glVertexAttribPointer(2,1,GLES30.GL_FLOAT,false,20,16);GLES30.glVertexAttribDivisor(2,1);GLES30.glBindVertexArray(0)}
 fun draw(d:DabBuffer,m:FloatArray){if(d.count==0)return;program?.use()?:return;GLES30.glUniformMatrix4fv(matrix,1,false,m,0);GLES30.glUniform3f(color,.12f,.25f,.95f);d.prepareForUpload();GLES30.glBindVertexArray(vao);GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER,instances);GLES30.glBufferSubData(GLES30.GL_ARRAY_BUFFER,0,d.count*20,d.floats);d.finishUpload();GLES30.glDrawArraysInstanced(GLES30.GL_TRIANGLE_STRIP,0,4,d.count);GLES30.glBindVertexArray(0)}
 fun drawInto(target:RenderTarget,w:Int,h:Int,m:FloatArray,d:DabBuffer,rgb:FloatArray){target.bind();GLES30.glViewport(0,0,w,h);GLES30.glEnable(GLES30.GL_BLEND);GLES30.glBlendFunc(GLES30.GL_ONE,GLES30.GL_ONE_MINUS_SRC_ALPHA);program?.use()?:return;GLES30.glUniformMatrix4fv(matrix,1,false,m,0);GLES30.glUniform3f(color,rgb[0],rgb[1],rgb[2]);d.prepareForUpload();GLES30.glBindVertexArray(vao);GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER,instances);GLES30.glBufferSubData(GLES30.GL_ARRAY_BUFFER,0,d.count*20,d.floats);d.finishUpload();GLES30.glDrawArraysInstanced(GLES30.GL_TRIANGLE_STRIP,0,4,d.count);GLES30.glBindVertexArray(0);GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER,0);GLES30.glDisable(GLES30.GL_BLEND)}
 fun release(){program?.release();program=null;if(instances!=0)GLES30.glDeleteBuffers(1,intArrayOf(instances),0);if(quad!=0)GLES30.glDeleteBuffers(1,intArrayOf(quad),0);if(vao!=0)GLES30.glDeleteVertexArrays(1,intArrayOf(vao),0);vao=0;quad=0;instances=0}
}
```

```
package com.wetinknext.engine.brush

import kotlin.math.PI
import kotlin.math.sqrt

class OneEuroFilter(private var minCutoff: Float = 1.6f, private var beta: Float = 0.25f, private var dCutoff: Float = 1f) {
    private var initialized = false; private var lastTimestampNanos = 0L
    private var x = 0f; private var y = 0f; private var dx = 0f; private var dy = 0f
    fun reset() { initialized = false; lastTimestampNanos = 0L; x = 0f; y = 0f; dx = 0f; dy = 0f }
    fun filter(timestampNanos: Long, rawX: Float, rawY: Float, out: FloatArray) {
        require(out.size >= 2)
        if (!initialized) { initialized = true; lastTimestampNanos = timestampNanos; x = rawX; y = rawY; out[0] = x; out[1] = y; return }
        var dt = (timestampNanos - lastTimestampNanos) / 1_000_000_000f
        if (dt < MIN_DT) dt = MIN_DT
        lastTimestampNanos = timestampNanos
        val rate = 1f / dt; val alphaD = alpha(rate, dCutoff)
        dx = alphaD * ((rawX - x) * rate) + (1f - alphaD) * dx; dy = alphaD * ((rawY - y) * rate) + (1f - alphaD) * dy
        val alpha = alpha(rate, (minCutoff + beta * sqrt(dx * dx + dy * dy)).coerceAtLeast(0.01f))
        x = alpha * rawX + (1f - alpha) * x; y = alpha * rawY + (1f - alpha) * y; out[0] = x; out[1] = y
    }
    private fun alpha(rate: Float, cutoff: Float): Float { val tau = 1f / (2f * PI.toFloat() * cutoff); return 1f / (1f + tau * rate) }
    private companion object { const val MIN_DT = 1f / 1000f }
}
```

```
package com.wetinknext.engine.brush

import kotlin.math.sqrt

class Stabilizer(private val filter: OneEuroFilter = OneEuroFilter()) {
    var strength = 0f; var x = 0f; private set; var y = 0f; private set; var velocity = 0f; private set
    private var hasLast = false; private var lastX = 0f; private var lastY = 0f; private var lastT = 0L; private val filtered = FloatArray(2)
    fun reset() { filter.reset(); x = 0f; y = 0f; velocity = 0f; hasLast = false; lastX = 0f; lastY = 0f; lastT = 0L }
    fun process(timestampNanos: Long, rawX: Float, rawY: Float) {
        if (strength <= 0f) { x = rawX; y = rawY } else { filter.filter(timestampNanos, rawX, rawY, filtered); x = filtered[0]; y = filtered[1] }
        if (hasLast) { var dt = (timestampNanos - lastT) / 1_000_000_000f; if (dt < 1f / 1000f) dt = 1f / 1000f; val d = sqrt((x-lastX)*(x-lastX)+(y-lastY)*(y-lastY)); velocity += 0.2f * (d / dt - velocity) }
        hasLast = true; lastX = x; lastY = y; lastT = timestampNanos
    }
}
```

```
package com.wetinknext.engine.brush

import com.wetinknext.engine.input.InputBatch
import kotlin.math.pow
import kotlin.math.sqrt

class StampEmitter(private val settings: BrushSettings) {
    private val stabilizer = Stabilizer(); private var active = false; private var pointerId = -1; private var hasLast = false
    private var lastX = 0f; private var lastY = 0f; private var lastPressure = 0f; private var carried = 0f; private var spacingPx = 1f
    fun reset() { active = false; pointerId = -1; hasLast = false; carried = 0f; stabilizer.reset() }
    fun begin(batch: InputBatch, out: DabBuffer) { reset(); if (batch.isEmpty()) return; stabilizer.strength = (settings.smoothing*.7f + settings.streamline*.3f).coerceIn(0f,1f); spacingPx = (if(settings.spacingUsesDiameter) settings.baseRadiusPx*2f*settings.spacing else settings.spacing).coerceIn(.75f,256f); val s=batch.samples[0]; active=true; pointerId=s.pointerId; stabilizer.process(s.timestampNanos,s.canvasX,s.canvasY); lastX=stabilizer.x;lastY=stabilizer.y;lastPressure=s.pressure;hasLast=true; addDab(lastX,lastY,lastPressure,out) }
    fun append(batch: InputBatch, out: DabBuffer) { if(!active)return; for(i in 0 until batch.sampleCount){val s=batch.samples[i];if(s.pointerId==pointerId){stabilizer.process(s.timestampNanos,s.canvasX,s.canvasY);addPoint(stabilizer.x,stabilizer.y,s.pressure,out)}} }
    fun finish(out: DabBuffer, cancel: Boolean) { if(active && !cancel && carried > .001f) addDab(lastX,lastY,lastPressure,out); reset() }
    private fun addPoint(x:Float,y:Float,p:Float,out:DabBuffer){ val dx=x-lastX;val dy=y-lastY;val dist=sqrt(dx*dx+dy*dy);if(!hasLast||dist<1e-5f){lastX=x;lastY=y;lastPressure=p;hasLast=true;return};var travelled=0f;var remaining=dist;while(carried+remaining>=spacingPx){val step=spacingPx-carried;travelled+=step;remaining-=step;val t=travelled/dist;addDab(lastX+dx*t,lastY+dy*t,lastPressure+(p-lastPressure)*t,out);carried=0f};carried+=remaining;lastX=x;lastY=y;lastPressure=p }
    private fun addDab(x:Float,y:Float,p:Float,out:DabBuffer){val pressure=p.coerceIn(0f,1f).let{if(settings.pressureGamma>0f)it.pow(settings.pressureGamma)else it};val size=if(settings.pressureToSize)settings.minSizeRatio+(1f-settings.minSizeRatio)*pressure else 1f;val alpha=if(settings.pressureToOpacity)pressure else 1f;out.add(x,y,(settings.baseRadiusPx*size).coerceAtLeast(.25f),0f,(settings.opacity*alpha).coerceIn(0f,1f))}
}
```

```kotlin
package com.wetinknext.engine.core

import java.util.concurrent.atomic.AtomicReference
import kotlin.math.min

/** Thread-safe camera state shared by the UI and GL threads. */
class Camera(initial: ViewTransform = ViewTransform()) {
    private val state = AtomicReference(initial)

    fun snapshot(): ViewTransform = state.get()

    fun set(transform: ViewTransform) {
        state.set(transform)
    }

    fun update(transform: (ViewTransform) -> ViewTransform) {
        while (true) {
            val current = state.get()
            if (state.compareAndSet(current, transform(current))) return
        }
    }

    fun reset() {
        state.set(ViewTransform())
    }

    fun fitCanvas(
        canvasWidth: Int,
        canvasHeight: Int,
        viewWidth: Int,
        viewHeight: Int,
        marginRatio: Float = 0.04f,
    ) {
        if (canvasWidth <= 0 || canvasHeight <= 0 || viewWidth <= 0 || viewHeight <= 0) return
        val usableWidth = viewWidth * (1f - marginRatio * 2f)
        val usableHeight = viewHeight * (1f - marginRatio * 2f)
        val scale = min(usableWidth / canvasWidth, usableHeight / canvasHeight)
            .coerceIn(ViewTransform.MIN_SCALE, ViewTransform.MAX_SCALE)
        state.set(
            ViewTransform(
                scale = scale,
                translateX = (viewWidth - canvasWidth * scale) * 0.5f,
                translateY = (viewHeight - canvasHeight * scale) * 0.5f,
            ),
        )
    }
}
```

```
package com.wetinknext.engine.core

import kotlin.math.ceil
import kotlin.math.floor
import kotlin.math.max
import kotlin.math.min

/** Reusable mutable dirty rectangle in canvas coordinates. */
class DirtyRect {
    var left = 0f; private set
    var top = 0f; private set
    var right = 0f; private set
    var bottom = 0f; private set
    var isEmpty = true; private set

    fun clear() {
        left = 0f; top = 0f; right = 0f; bottom = 0f; isEmpty = true
    }

    fun include(x: Float, y: Float, radius: Float) = include(x - radius, y - radius, x + radius, y + radius)

    fun include(left: Float, top: Float, right: Float, bottom: Float) {
        if (right < left || bottom < top) return
        if (isEmpty) {
            this.left = left; this.top = top; this.right = right; this.bottom = bottom; isEmpty = false
            return
        }
        this.left = min(this.left, left); this.top = min(this.top, top)
        this.right = max(this.right, right); this.bottom = max(this.bottom, bottom)
    }

    fun include(other: DirtyRect) {
        if (!other.isEmpty) include(other.left, other.top, other.right, other.bottom)
    }

    fun expand(pixels: Float) {
        if (!isEmpty && pixels > 0f) {
            left -= pixels; top -= pixels; right += pixels; bottom += pixels
        }
    }

    fun clamp(canvasWidth: Float, canvasHeight: Float) {
        if (isEmpty) return
        left = left.coerceIn(0f, canvasWidth); top = top.coerceIn(0f, canvasHeight)
        right = right.coerceIn(0f, canvasWidth); bottom = bottom.coerceIn(0f, canvasHeight)
        if (right <= left || bottom <= top) clear()
    }

    fun copyTo(out: DirtyRect) {
        out.left = left; out.top = top; out.right = right; out.bottom = bottom; out.isEmpty = isEmpty
    }

    fun toPixelBounds(out: IntArray) {
        require(out.size >= 4)
        if (isEmpty) {
            out[0] = 0; out[1] = 0; out[2] = 0; out[3] = 0
            return
        }
        out[0] = floor(left).toInt(); out[1] = floor(top).toInt()
        out[2] = ceil(right).toInt(); out[3] = ceil(bottom).toInt()
    }
}
```

```
package com.wetinknext.engine.core

import android.opengl.GLES30
import android.opengl.GLSurfaceView
import android.view.MotionEvent
import com.wetinknext.engine.brush.BrushSettings
import com.wetinknext.engine.brush.ColorSpaces
import com.wetinknext.engine.brush.DabBuffer
import com.wetinknext.engine.brush.DabRenderer
import com.wetinknext.engine.brush.StampEmitter
import com.wetinknext.engine.gl.CanvasGeometry
import com.wetinknext.engine.gl.GlCaps
import com.wetinknext.engine.gl.GlCheck
import com.wetinknext.engine.gl.GlProgram
import com.wetinknext.engine.gl.ShaderLib
import com.wetinknext.engine.gl.RenderTarget
import javax.microedition.khronos.egl.EGLConfig
import javax.microedition.khronos.opengles.GL10
import java.util.concurrent.ArrayBlockingQueue
import java.util.concurrent.atomic.AtomicBoolean
import com.wetinknext.engine.input.InputAction
import com.wetinknext.engine.input.InputBatch
import com.wetinknext.engine.input.InputBatchPool
import com.wetinknext.engine.input.StrokeInputCapturer

class EngineRenderer(
    private val documentWidth: Int = DEFAULT_CANVAS_WIDTH,
    private val documentHeight: Int = DEFAULT_CANVAS_HEIGHT,
) : GLSurfaceView.Renderer {
    private val paintEngine = PaintEngine()
    private var caps: GlCaps? = null
    private var presentProgram: GlProgram? = null
    private var geometry: CanvasGeometry? = null
    private var screenWidth = 1
    private var screenHeight = 1
    private var uTexture = -1
    private var uCanvasToClip = -1
    private var uCanvasSize = -1
    private val presentMatrix = FloatArray(16)
    private val canvasToFboMatrix = FloatArray(16)
    private val inputPool = InputBatchPool(64, 256)
    private val inputQueue = ArrayBlockingQueue<InputBatch>(64)
    private val inputCapturer = StrokeInputCapturer(paintEngine.camera, inputPool, inputQueue)
    private val dabBuffer = DabBuffer()
    private val defaultBrush = BrushSettings(name="Round Opaque", baseRadiusPx=16f, spacing=.12f, colorArgb=0xFF1B1F24L, smoothi
ng=.25f, streamline=.05f, pressureToOpacity=false, pressureGamma=1.1f, minSizeRatio=.12f)
    private val stampEmitter = StampEmitter(defaultBrush)
    private val strokeTarget = RenderTarget()
    private val strokeColor = FloatArray(3)
    private var dabRenderer: DabRenderer? = null
    private var strokeActive = false
    private var previewVisible = false
    private val cancelRequested = AtomicBoolean(false)

    fun onTouchEvent(event: MotionEvent): Boolean = inputCapturer.onTouchEvent(event)

    override fun onSurfaceCreated(gl: GL10?, config: EGLConfig?) {
        releaseGlObjects()
        val nextCaps = GlCaps.query(); caps = nextCaps
        presentProgram = GlProgram(ShaderLib.canvasPresentVertex, ShaderLib.strokeCompositeFragment)
        paintEngine.create(nextCaps, documentWidth, documentHeight)
        paintEngine.clearCanvas()
        strokeTarget.create(documentWidth, documentHeight, nextCaps.supportsHalfFloatColorBuffer)
        strokeTarget.clear(0f,0f,0f,0f)
        ViewTransform.buildCanvasToFbo(documentWidth.toFloat(), documentHeight.toFloat(), canvasToFboMatrix)
        geometry = CanvasGeometry().also { it.create(documentWidth, documentHeight) }
        dabRenderer = DabRenderer(dabBuffer.capacity).also { it.create() }
        presentProgram!!.use()
        uTexture = GLES30.glGetUniformLocation(presentProgram!!.id, "uCanvasTex")
        uCanvasToClip = GLES30.glGetUniformLocation(presentProgram!!.id, "uCanvasToClip")
        uCanvasSize = GLES30.glGetUniformLocation(presentProgram!!.id, "uCanvasSize")
        check(uTexture >= 0 && uCanvasToClip >= 0 && uCanvasSize >= 0) { "P3 present shader uniforms missing" }
        GlCheck.noError("P3 surface creation")
    }

    override fun onSurfaceChanged(gl: GL10?, width: Int, height: Int) {
        screenWidth = width.coerceAtLeast(1); screenHeight = height.coerceAtLeast(1)
        GLES30.glViewport(0, 0, screenWidth, screenHeight)
        paintEngine.camera.fitCanvas(paintEngine.canvasWidth, paintEngine.canvasHeight, screenWidth, screenHeight)
        GlCheck.noError("P3 surface changed")
    }

    override fun onDrawFrame(gl: GL10?) {
        if (cancelRequested.compareAndSet(true, false)) resetStroke()
        drainInput()
        if(strokeActive && dabBuffer.count>0){strokeTarget.clear(0f,0f,0f,0f);dabRenderer?.drawInto(strokeTarget,paintEngine.can
vasWidth,paintEngine.canvasHeight,canvasToFboMatrix,dabBuffer,strokeColor)}
        GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
        GLES30.glViewport(0, 0, screenWidth, screenHeight)
```

```
        GLES30.glDisable(GLES30.GL_BLEND); GLES30.glDisable(GLES30.GL_SCISSOR_TEST)
        GLES30.glClearColor(0.08f, 0.09f, 0.12f, 1f); GLES30.glClear(GLES30.GL_COLOR_BUFFER_BIT)
        val program = presentProgram ?: return
        program.use()
        paintEngine.camera.snapshot().buildCanvasToClip(screenWidth.toFloat(), screenHeight.toFloat(), presentMatrix)
        GLES30.glUniformMatrix4fv(uCanvasToClip, 1, false, presentMatrix, 0)
        GLES30.glUniform2f(uCanvasSize, paintEngine.canvasWidth.toFloat(), paintEngine.canvasHeight.toFloat())
        GLES30.glActiveTexture(GLES30.GL_TEXTURE0); GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, paintEngine.canvasTarget.textureId)
        GLES30.glUniform1i(uTexture, 0)
        val strokeTex=GLES30.glGetUniformLocation(program.id,"uStrokeTex");val strokeActiveUniform=GLES30.glGetUniformLocation(program.id,"uStrokeActive")
        GLES30.glActiveTexture(GLES30.GL_TEXTURE1);GLES30.glBindTexture(GLES30.GL_TEXTURE_2D,strokeTarget.textureId);GLES30.glUniform1i(strokeTex,1);GLES30.glUniform1i(strokeActiveUniform,if(strokeActive)1 else 0);geometry?.draw();GLES30.glBindTexture(GLES30.GL_TEXTURE_2D,0);GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
    }

    fun cancelActiveStroke() { cancelRequested.set(true) }
    fun releaseGlObjects() {
        drainInput(); resetStroke()
        dabRenderer?.release(); dabRenderer = null
        strokeTarget.release()
        geometry?.release(); geometry = null
        presentProgram?.release(); presentProgram = null
        paintEngine.release(); caps = null
        uTexture = -1; uCanvasToClip = -1; uCanvasSize = -1
    }

    private fun drainInput() {
        while (true) {
            val batch = inputQueue.poll() ?: return
            when (batch.action) {
                InputAction.DOWN -> if (!batch.isEmpty()) { resetStroke(); strokeActive = true; previewVisible = true; ColorSpaces.srgb8ToLinear(defaultBrush.colorArgb,strokeColor); stampEmitter.begin(batch, dabBuffer) }
                InputAction.MOVE -> if (strokeActive) stampEmitter.append(batch, dabBuffer)
                InputAction.UP -> if (strokeActive) { stampEmitter.finish(dabBuffer, false); dabRenderer?.drawInto(paintEngine.canvasTarget,paintEngine.canvasWidth,paintEngine.canvasHeight,canvasToFboMatrix,dabBuffer,strokeColor); dabBuffer.clear(); strokeActive = false }
                InputAction.CANCEL -> resetStroke()
            }
            inputPool.release(batch)
        }
    }

    private fun resetStroke() { strokeActive = false; previewVisible = false; stampEmitter.reset(); dabBuffer.clear() }

    companion object { const val DEFAULT_CANVAS_WIDTH = 1500; const val DEFAULT_CANVAS_HEIGHT = 2000 }
}
```

```
package com.wetinknext.engine.core

import com.wetinknext.engine.gl.GlCaps
import com.wetinknext.engine.gl.RenderTarget

/** Document state that is exclusively owned by the GLSurfaceView render thread. */
class PaintEngine {
    val camera = Camera()
    val canvasTarget = RenderTarget()
    var canvasWidth = 1; private set
    var canvasHeight = 1; private set
    var initialized = false; private set

    fun create(caps: GlCaps, width: Int, height: Int) {
        require(width > 0 && height > 0)
        release()
        canvasWidth = width; canvasHeight = height
        val useHalfFloat = caps.supportsHalfFloatColorBuffer && canvasTarget.probeHalfFloatColorBuffer()
        canvasTarget.create(width, height, useHalfFloat)
        initialized = true
    }

    fun resize(caps: GlCaps, width: Int, height: Int) {
        if (!initialized || canvasWidth != width || canvasHeight != height) create(caps, width, height)
    }

    fun clearCanvas() {
        check(initialized)
        canvasTarget.clear(1f, 1f, 1f, 1f)
    }

    fun release() {
        canvasTarget.release(); initialized = false; canvasWidth = 1; canvasHeight = 1
    }
}
```

```kotlin
package com.wetinknext.engine.core

import android.content.Context
import android.opengl.GLSurfaceView
import android.util.AttributeSet
import android.view.MotionEvent

class PaintSurfaceView @JvmOverloads constructor(
    context: Context,
    attrs: AttributeSet? = null,
) : GLSurfaceView(context, attrs) {
    private val engineRenderer = EngineRenderer()

    init {
        setEGLContextClientVersion(3)
        setRenderer(engineRenderer)
        renderMode = RENDERMODE_CONTINUOUSLY
    }

    override fun onTouchEvent(event: MotionEvent): Boolean =
        engineRenderer.onTouchEvent(event) || super.onTouchEvent(event)

    override fun onPause() {
        engineRenderer.cancelActiveStroke()
        super.onPause()
    }

    override fun onDetachedFromWindow() {
        queueEvent { engineRenderer.releaseGlObjects() }
        super.onDetachedFromWindow()
    }
}
```

```
package com.wetinknext.engine.core

import kotlin.math.abs
import kotlin.math.cos
import kotlin.math.sin

/** Canonical canvas-to-view-local-screen transformation. */
data class ViewTransform(
    val translateX: Float = 0f,
    val translateY: Float = 0f,
    val scale: Float = 1f,
    val rotationRad: Float = 0f,
    val flipX: Boolean = false,
) {
    private val flipSign: Float
        get() = if (flipX) -1f else 1f

    val m00: Float get() = cos(rotationRad) * scale * flipSign
    val m01: Float get() = -sin(rotationRad) * scale
    val m10: Float get() = sin(rotationRad) * scale * flipSign
    val m11: Float get() = cos(rotationRad) * scale

    fun canvasToScreen(canvasX: Float, canvasY: Float, out: FloatArray) {
        require(out.size >= 2)
        out[0] = m00 * canvasX + m01 * canvasY + translateX
        out[1] = m10 * canvasX + m11 * canvasY + translateY
    }

    fun screenToCanvas(screenX: Float, screenY: Float, out: FloatArray) {
        require(out.size >= 2)
        val determinant = m00 * m11 - m01 * m10
        if (abs(determinant) < 1e-8f) {
            out[0] = 0f
            out[1] = 0f
            return
        }
        val inverseDeterminant = 1f / determinant
        val dx = screenX - translateX
        val dy = screenY - translateY
        out[0] = (m11 * dx - m01 * dy) * inverseDeterminant
        out[1] = (-m10 * dx + m00 * dy) * inverseDeterminant
    }

    fun zoomAround(anchorX: Float, anchorY: Float, newScale: Float): ViewTransform {
        val safeScale = newScale.coerceIn(MIN_SCALE, MAX_SCALE)
        val factor = safeScale / scale
        return copy(
            scale = safeScale,
            translateX = anchorX - (anchorX - translateX) * factor,
            translateY = anchorY - (anchorY - translateY) * factor,
        )
    }

    fun rotateAround(anchorX: Float, anchorY: Float, deltaRad: Float): ViewTransform {
        val c = cos(deltaRad)
        val s = sin(deltaRad)
        val dx = translateX - anchorX
        val dy = translateY - anchorY
        return copy(
            rotationRad = rotationRad + deltaRad,
            translateX = anchorX + dx * c - dy * s,
            translateY = anchorY + dx * s + dy * c,
        )
    }

    /** Builds a column-major matrix for canvas to OpenGL clip coordinates. */
    fun buildCanvasToClip(viewWidth: Float, viewHeight: Float, out: FloatArray) {
        require(out.size >= 16)
        require(viewWidth > 0f)
        require(viewHeight > 0f)
        val sx = 2f / viewWidth
        val sy = -2f / viewHeight
        out[0] = m00 * sx; out[1] = m10 * sy; out[2] = 0f; out[3] = 0f
        out[4] = m01 * sx; out[5] = m11 * sy; out[6] = 0f; out[7] = 0f
        out[8] = 0f; out[9] = 0f; out[10] = 1f; out[11] = 0f
        out[12] = translateX * sx - 1f; out[13] = translateY * sy + 1f
        out[14] = 0f; out[15] = 1f
    }

    companion object {
        const val MIN_SCALE = 0.02f
        const val MAX_SCALE = 64f

        fun buildCanvasToFbo(canvasWidth: Float, canvasHeight: Float, out: FloatArray) {
            require(out.size >= 16); require(canvasWidth > 0f); require(canvasHeight > 0f)
            out[0]=2f/canvasWidth;out[1]=0f;out[2]=0f;out[3]=0f;out[4]=0f;out[5]=2f/canvasHeight;out[6]=0f;out[7]=0f;out[8]=0f;o
ut[9]=0f;out[10]=1f;out[11]=0f;out[12]=-1f;out[13]=-1f;out[14]=0f;out[15]=1f
        }
    }
```

```
89  }
```

```
package com.wetinknext.engine.gl

import android.opengl.GLES30
import java.nio.ByteBuffer
import java.nio.ByteOrder

/** A document-sized quad; GL resources are owned by the render thread. */
class CanvasGeometry {
    private var vaoId = 0
    private var vboId = 0

    fun create(width: Int, height: Int) {
        release()
        val vertices = floatArrayOf(0f, 0f, width.toFloat(), 0f, 0f, height.toFloat(), width.toFloat(), height.toFloat())
        val data = ByteBuffer.allocateDirect(vertices.size * Float.SIZE_BYTES).order(ByteOrder.nativeOrder()).asFloatBuffer()
        data.put(vertices).position(0)
        val ids = IntArray(1)
        GLES30.glGenVertexArrays(1, ids, 0); vaoId = ids[0]
        GLES30.glGenBuffers(1, ids, 0); vboId = ids[0]
        GLES30.glBindVertexArray(vaoId); GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, vboId)
        GLES30.glBufferData(GLES30.GL_ARRAY_BUFFER, vertices.size * Float.SIZE_BYTES, data, GLES30.GL_STATIC_DRAW)
        GLES30.glEnableVertexAttribArray(0); GLES30.glVertexAttribPointer(0, 2, GLES30.GL_FLOAT, false, 0, 0)
        GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, 0); GLES30.glBindVertexArray(0)
    }

    fun draw() {
        check(vaoId != 0)
        GLES30.glBindVertexArray(vaoId); GLES30.glDrawArrays(GLES30.GL_TRIANGLE_STRIP, 0, 4); GLES30.glBindVertexArray(0)
    }

    fun release() {
        if (vboId != 0) GLES30.glDeleteBuffers(1, intArrayOf(vboId), 0)
        if (vaoId != 0) GLES30.glDeleteVertexArrays(1, intArrayOf(vaoId), 0)
        vaoId = 0; vboId = 0
    }
}
```

```
package com.wetinknext.engine.gl

import android.opengl.GLES30
import java.nio.ByteBuffer
import java.nio.ByteOrder

/** A single cached fullscreen triangle for future present passes. */
class GeometryCache {
    private var vaoId = 0
    private var vboId = 0
    fun create() {
        if (vaoId != 0) return
        val vertices = floatArrayOf(-1f, -1f, 3f, -1f, -1f, 3f)
        val data = ByteBuffer.allocateDirect(vertices.size * Float.SIZE_BYTES).order(ByteOrder.nativeOrder()).asFloatBuffer()
        data.put(vertices).position(0)
        val ids = IntArray(1); GLES30.glGenVertexArrays(1, ids, 0); vaoId = ids[0]
        GLES30.glGenBuffers(1, ids, 0); vboId = ids[0]
        GLES30.glBindVertexArray(vaoId); GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, vboId)
        GLES30.glBufferData(GLES30.GL_ARRAY_BUFFER, vertices.size * Float.SIZE_BYTES, data, GLES30.GL_STATIC_DRAW)
        GLES30.glEnableVertexAttribArray(0); GLES30.glVertexAttribPointer(0, 2, GLES30.GL_FLOAT, false, 0, 0)
        GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, 0); GLES30.glBindVertexArray(0)
    }
    fun drawFullscreenTriangle() { check(vaoId != 0); GLES30.glBindVertexArray(vaoId); GLES30.glDrawArrays(GLES30.GL_TRIANGLES, 0, 3); GLES30.glBindVertexArray(0) }
    fun release() { if (vboId != 0) GLES30.glDeleteBuffers(1, intArrayOf(vboId), 0); if (vaoId != 0) GLES30.glDeleteVertexArrays(1, intArrayOf(vaoId), 0); vboId = 0; vaoId = 0 }
}
```

```kotlin
package com.wetinknext.engine.gl

import android.opengl.GLES30

/** Capabilities queried only after a GLES 3 context is current. */
data class GlCaps(
    val renderer: String,
    val version: String,
    val supportsHalfFloatColorBuffer: Boolean,
) {
    companion object {
        fun query(): GlCaps {
            val extensions = GLES30.glGetString(GLES30.GL_EXTENSIONS).orEmpty()
            return GlCaps(
                renderer = GLES30.glGetString(GLES30.GL_RENDERER).orEmpty(),
                version = GLES30.glGetString(GLES30.GL_VERSION).orEmpty(),
                supportsHalfFloatColorBuffer = extensions.contains("EXT_color_buffer_half_float"),
            )
        }
    }
}
```

```
package com.wetinknext.engine.gl

import android.opengl.GLES30

/** GL error checks intended for setup boundaries, never the render hot path. */
object GlCheck {
    fun noError(label: String) {
        val error = GLES30.glGetError()
        check(error == GLES30.GL_NO_ERROR) { "$label: GL error 0x${error.toString(16)}" }
    }

    fun framebufferComplete(label: String) {
        val status = GLES30.glCheckFramebufferStatus(GLES30.GL_FRAMEBUFFER)
        check(status == GLES30.GL_FRAMEBUFFER_COMPLETE) {
            "$label: framebuffer incomplete (0x${status.toString(16)})"
        }
    }
}
```

```
package com.wetinknext.engine.gl

import android.opengl.GLES30

/** Linked GLSL program. Construction and release must run on the GL thread. */
class GlProgram(vertexSource: String, fragmentSource: String) {
    val id: Int = GLES30.glCreateProgram().also { program ->
        check(program != 0) { "Unable to create GL program" }
        val vertex = compile(GLES30.GL_VERTEX_SHADER, vertexSource)
        val fragment = compile(GLES30.GL_FRAGMENT_SHADER, fragmentSource)
        GLES30.glAttachShader(program, vertex); GLES30.glAttachShader(program, fragment); GLES30.glLinkProgram(program)
        val linked = IntArray(1); GLES30.glGetProgramiv(program, GLES30.GL_LINK_STATUS, linked, 0)
        GLES30.glDeleteShader(vertex); GLES30.glDeleteShader(fragment)
        check(linked[0] == GLES30.GL_TRUE) { "Program link failed: ${GLES30.glGetProgramInfoLog(program)}" }
    }
    fun use() = GLES30.glUseProgram(id)
    fun release() = GLES30.glDeleteProgram(id)

    private fun compile(type: Int, source: String): Int = GLES30.glCreateShader(type).also { shader ->
        check(shader != 0) { "Unable to create shader" }
        GLES30.glShaderSource(shader, source); GLES30.glCompileShader(shader)
        val compiled = IntArray(1); GLES30.glGetShaderiv(shader, GLES30.GL_COMPILE_STATUS, compiled, 0)
        check(compiled[0] == GLES30.GL_TRUE) { "Shader compile failed: ${GLES30.glGetShaderInfoLog(shader)}" }
    }
}
```

```
package com.wetinknext.engine.gl

import android.opengl.GLES30

/** Texture-backed FBO with RGBA16F probing and a guaranteed RGBA8 fallback. */
class RenderTarget {
    var framebufferId: Int = 0
        private set
    var textureId: Int = 0
        private set
    var width: Int = 0
        private set
    var height: Int = 0
        private set
    var usesHalfFloat: Boolean = false
        private set

    fun create(width: Int, height: Int, preferHalfFloat: Boolean) {
        require(width > 0 && height > 0)
        release()
        if (preferHalfFloat && createInternal(width, height, true)) return
        check(createInternal(width, height, false)) { "RGBA8 render target creation failed" }
    }

    /** A 4x4 completeness probe used before allocating a document-sized target. */
    fun probeHalfFloatColorBuffer(): Boolean {
        release()
        val result = createInternal(4, 4, true)
        release()
        return result
    }

    fun bind() {
        check(framebufferId != 0) { "RenderTarget is not created" }
        GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, framebufferId)
    }

    fun clear(red: Float, green: Float, blue: Float, alpha: Float) {
        bind()
        GLES30.glDisable(GLES30.GL_SCISSOR_TEST)
        GLES30.glDisable(GLES30.GL_BLEND)
        GLES30.glClearColor(red, green, blue, alpha)
        GLES30.glClear(GLES30.GL_COLOR_BUFFER_BIT)
    }

    fun release() {
        if (framebufferId != 0) GLES30.glDeleteFramebuffers(1, intArrayOf(framebufferId), 0)
        if (textureId != 0) GLES30.glDeleteTextures(1, intArrayOf(textureId), 0)
        framebufferId = 0; textureId = 0; width = 0; height = 0; usesHalfFloat = false
    }

    private fun createInternal(targetWidth: Int, targetHeight: Int, halfFloat: Boolean): Boolean {
        val textures = IntArray(1)
        val framebuffers = IntArray(1)
        GLES30.glGenTextures(1, textures, 0)
        GLES30.glGenFramebuffers(1, framebuffers, 0)
        val texture = textures[0]
        val framebuffer = framebuffers[0]
        if (texture == 0 || framebuffer == 0) return false
        GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, texture)
        GLES30.glTexParameteri(GLES30.GL_TEXTURE_2D, GLES30.GL_TEXTURE_MIN_FILTER, GLES30.GL_LINEAR)
        GLES30.glTexParameteri(GLES30.GL_TEXTURE_2D, GLES30.GL_TEXTURE_MAG_FILTER, GLES30.GL_LINEAR)
        GLES30.glTexParameteri(GLES30.GL_TEXTURE_2D, GLES30.GL_TEXTURE_WRAP_S, GLES30.GL_CLAMP_TO_EDGE)
        GLES30.glTexParameteri(GLES30.GL_TEXTURE_2D, GLES30.GL_TEXTURE_WRAP_T, GLES30.GL_CLAMP_TO_EDGE)
        GLES30.glTexImage2D(GLES30.GL_TEXTURE_2D, 0, if (halfFloat) GLES30.GL_RGBA16F else GLES30.GL_RGBA8,
            targetWidth, targetHeight, 0, GLES30.GL_RGBA, if (halfFloat) GLES30.GL_HALF_FLOAT else GLES30.GL_UNSIGNED_BYTE, null
)
        GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, framebuffer)
        GLES30.glFramebufferTexture2D(GLES30.GL_FRAMEBUFFER, GLES30.GL_COLOR_ATTACHMENT0, GLES30.GL_TEXTURE_2D, texture, 0)
        val complete = GLES30.glCheckFramebufferStatus(GLES30.GL_FRAMEBUFFER) == GLES30.GL_FRAMEBUFFER_COMPLETE
        GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
        GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
        if (!complete) {
            GLES30.glDeleteFramebuffers(1, framebuffers, 0); GLES30.glDeleteTextures(1, textures, 0)
            return false
        }
        framebufferId = framebuffer; textureId = texture; width = targetWidth; height = targetHeight; usesHalfFloat = halfFloat
        return true
    }
}
```

```
package com.wetinknext.engine.gl

object ShaderLib {
    const val dabVertex = """#version 300 es
        layout(location = 0) in vec2 aCorner;
        layout(location = 1) in vec4 iDab0;
        layout(location = 2) in float iAlpha;
        uniform mat4 uCanvasToClip;
        out vec2 vLocal; flat out float vAlpha;
        void main(){ float c=cos(iDab0.w), s=sin(iDab0.w); vec2 r=vec2(c*aCorner.x-s*aCorner.y,s*aCorner.x+c*aCorner.y); vLocal=aCorner;vAlpha=iAlpha;gl_Position=uCanvasToClip*vec4(iDab0.xy+r*iDab0.z,0.,1.); }
    """
    const val dabFragment = """#version 300 es
        precision highp float; in vec2 vLocal; flat in float vAlpha; uniform vec3 uColorLinear; out vec4 fragColor;
        void main(){ float r=length(vLocal), aa=max(fwidth(r),.001), a=vAlpha*(1.-smoothstep(1.-aa,1.+aa,r));fragColor=vec4(uColorLinear*a,a); }
    """
    const val strokeCompositeFragment = """#version 300 es
        precision highp float; in vec2 vUv; uniform sampler2D uCanvasTex; uniform sampler2D uStrokeTex; uniform int uStrokeActive; out vec4 fragColor;
        vec3 toSrgb(vec3 c){c=clamp(c,0.,1.);return mix(c*12.92,1.055*pow(c,vec3(1./2.4))-.055,step(vec3(.0031308),c));}
        vec3 unp(vec4 c){return c.a<=.0001?vec3(0.):c.rgb/c.a;}
        void main(){vec4 c=texture(uCanvasTex,vUv);vec4 s=texture(uStrokeTex,vUv);float a=uStrokeActive==1?s.a+c.a*(1.-s.a):c.a;vec3 rgb=uStrokeActive==1&&a>.0001?(unp(s)*s.a+unp(c)*c.a*(1.-s.a))/a:unp(c);fragColor=vec4(toSrgb(rgb),1.);}
    """
    const val canvasPresentVertex = """#version 300 es
        layout(location = 0) in vec2 aPosition;
        uniform mat4 uCanvasToClip;
        uniform vec2 uCanvasSize;
        out vec2 vUv;
        void main() {
            vUv = aPosition / uCanvasSize;
            gl_Position = uCanvasToClip * vec4(aPosition, 0.0, 1.0);
        }
    """
    const val presentFragment = """#version 300 es
        precision highp float;
        in vec2 vUv;
        uniform sampler2D uTexture;
        out vec4 fragColor;
        void main() { fragColor = texture(uTexture, vUv); }
    """
    const val fullscreenVertex = """#version 300 es
        layout(location = 0) in vec2 aPosition;
        out vec2 vUv;
        void main() { vUv = aPosition * 0.5 + 0.5; gl_Position = vec4(aPosition, 0.0, 1.0); }
    """
    const val solidFragment = """#version 300 es
        precision highp float;
        in vec2 vUv;
        out vec4 fragColor;
        void main() { fragColor = vec4(vUv, 0.0, 1.0); }
    """
}
```

app/src/main/java/com/wetinknext/engine/input/InputAction.kt

```
package com.wetinknext.engine.input

enum class InputAction {
    DOWN,
    MOVE,
    UP,
    CANCEL,
}
```

```kotlin
package com.wetinknext.engine.input

/** Fixed-capacity container for passing input from the UI thread to the GL thread. */
class InputBatch(val maxSamples: Int) {
    val samples = Array(maxSamples) { InputSample() }

    var action: InputAction = InputAction.MOVE
        private set
    var sampleCount: Int = 0
        private set
    var prediction: Boolean = false
        private set

    fun begin(action: InputAction, prediction: Boolean = false) {
        this.action = action
        this.prediction = prediction
        sampleCount = 0
    }

    fun addSample(
        canvasX: Float, canvasY: Float, pressure: Float, tiltX: Float, tiltY: Float,
        orientationRad: Float, timestampNanos: Long, pointerId: Int, tool: PointerTool,
        historical: Boolean,
    ): Boolean {
        if (sampleCount >= maxSamples) return false
        samples[sampleCount].set(
            canvasX, canvasY, pressure.coerceIn(0f, 1f), tiltX.coerceIn(-1f, 1f),
            tiltY.coerceIn(-1f, 1f), orientationRad, timestampNanos, pointerId, tool, historical,
        )
        sampleCount += 1
        return true
    }

    fun isEmpty(): Boolean = sampleCount == 0

    fun clear() {
        for (index in 0 until sampleCount) samples[index].clear()
        sampleCount = 0
        prediction = false
        action = InputAction.MOVE
    }
}
```

```kotlin
package com.wetinknext.engine.input

import java.util.concurrent.atomic.AtomicInteger
import java.util.concurrent.atomic.AtomicReferenceArray

/** Fixed-size pool that returns null instead of allocating when exhausted. */
class InputBatchPool(
    batchCount: Int = DEFAULT_BATCH_COUNT,
    maxSamplesPerBatch: Int = DEFAULT_MAX_SAMPLES,
) {
    private val batches = AtomicReferenceArray<InputBatch>(batchCount)
    private val available = AtomicInteger(batchCount)

    init {
        require(batchCount > 0)
        require(maxSamplesPerBatch > 0)
        for (index in 0 until batchCount) batches.set(index, InputBatch(maxSamplesPerBatch))
    }

    fun acquire(): InputBatch? {
        while (true) {
            val current = available.get()
            if (current <= 0) return null
            if (available.compareAndSet(current, current - 1)) {
                for (index in 0 until batches.length()) {
                    val batch = batches.getAndSet(index, null)
                    if (batch != null) return batch
                }
                available.incrementAndGet()
                return null
            }
        }
    }

    fun release(batch: InputBatch) {
        batch.clear()
        for (index in 0 until batches.length()) {
            if (batches.compareAndSet(index, null, batch)) {
                available.incrementAndGet()
                return
            }
        }
        error("InputBatchPool overflow: released batch does not belong to pool")
    }

    val freeCount: Int get() = available.get()

    companion object {
        const val DEFAULT_BATCH_COUNT = 96
        const val DEFAULT_MAX_SAMPLES = 64
    }
}
```

```
package com.wetinknext.engine.input

/** Mutable input sample allocated only while a batch pool is initialized. */
class InputSample {
    var canvasX = 0f
    var canvasY = 0f
    var pressure = 0f
    var tiltX = 0f
    var tiltY = 0f
    var orientationRad = 0f
    var timestampNanos = 0L
    var pointerId = -1
    var tool = PointerTool.UNKNOWN
    var historical = false

    fun set(
        canvasX: Float, canvasY: Float, pressure: Float, tiltX: Float, tiltY: Float,
        orientationRad: Float, timestampNanos: Long, pointerId: Int, tool: PointerTool,
        historical: Boolean,
    ) {
        this.canvasX = canvasX; this.canvasY = canvasY; this.pressure = pressure
        this.tiltX = tiltX; this.tiltY = tiltY; this.orientationRad = orientationRad
        this.timestampNanos = timestampNanos; this.pointerId = pointerId; this.tool = tool
        this.historical = historical
    }

    fun clear() {
        canvasX = 0f; canvasY = 0f; pressure = 0f; tiltX = 0f; tiltY = 0f
        orientationRad = 0f; timestampNanos = 0L; pointerId = -1
        tool = PointerTool.UNKNOWN; historical = false
    }
}
```

app/src/main/java/com/wetinknext/engine/input/PointerTool.kt

```kotlin
package com.wetinknext.engine.input

enum class PointerTool {
    FINGER,
    STYLUS,
    ERASER,
    UNKNOWN,
}
```

```
package com.wetinknext.engine.input

import android.view.MotionEvent
import com.wetinknext.engine.core.Camera
import java.util.concurrent.ArrayBlockingQueue

/** Converts View-local MotionEvent samples to canvas coordinates on the UI thread. */
class StrokeInputCapturer(private val camera: Camera, private val pool: InputBatchPool, private val queue: ArrayBlockingQueue<InputBatch>) {
    private var activePointerId = -1; private val canvasPoint = FloatArray(2)
    fun onTouchEvent(event: MotionEvent): Boolean = when(event.actionMasked) {
        MotionEvent.ACTION_DOWN, MotionEvent.ACTION_POINTER_DOWN -> { val i=if(event.actionMasked==MotionEvent.ACTION_DOWN) 0 else event.actionIndex; activePointerId=event.getPointerId(i); enqueue(event,InputAction.DOWN,i,false) }
        MotionEvent.ACTION_MOVE -> { val i=event.findPointerIndex(activePointerId); i >= 0 && enqueue(event,InputAction.MOVE,i,true) }
        MotionEvent.ACTION_UP, MotionEvent.ACTION_POINTER_UP -> { val i=if(event.actionMasked==MotionEvent.ACTION_UP) 0 else event.actionIndex; if(event.getPointerId(i)!=activePointerId) true else { activePointerId=-1; enqueue(event,InputAction.UP,i,false) } }
        MotionEvent.ACTION_CANCEL -> { activePointerId=-1; enqueueEmpty(InputAction.CANCEL) }
        else -> false
    }
    private fun enqueue(event:MotionEvent, action:InputAction, index:Int, historical:Boolean):Boolean { val batch=pool.acquire()?:return false;batch.begin(action);val t=camera.snapshot();val id=event.getPointerId(index);val tool=tool(event,index);if(historical)for(h in 0 until event.historySize){t.screenToCanvas(event.getHistoricalX(index,h),event.getHistoricalY(index,h),canvasPoint);if(!batch.addSample(canvasPoint[0],canvasPoint[1],pressure(event,tool,index,h),0f,0f,orientation(event,tool,index,h),event.getHistoricalEventTime(h)*1_000_000L,id,tool,true))break};t.screenToCanvas(event.getX(index),event.getY(index),canvasPoint);batch.addSample(canvasPoint[0],canvasPoint[1],pressure(event,tool,index,-1),0f,0f,orientation(event,tool,index,-1),event.eventTime*1_000_000L,id,tool,false);if(!queue.offer(batch)){pool.release(batch);return false};return true }
    private fun enqueueEmpty(action:InputAction):Boolean { val batch=pool.acquire()?:return false;batch.begin(action);if(!queue.offer(batch)){pool.release(batch);return false};return true }
    private fun tool(e:MotionEvent,i:Int)=when(e.getToolType(i)){MotionEvent.TOOL_TYPE_STYLUS->PointerTool.STYLUS;MotionEvent.TOOL_TYPE_ERASER->PointerTool.ERASER;MotionEvent.TOOL_TYPE_FINGER->PointerTool.FINGER;else->PointerTool.UNKNOWN}
    private fun pressure(e:MotionEvent,t:PointerTool,i:Int,h:Int):Float=if(t==PointerTool.FINGER||t==PointerTool.UNKNOWN).62f else (if(h>=0)e.getHistoricalPressure(i,h) else e.getPressure(i)).coerceIn(0f,1f)
    private fun orientation(e:MotionEvent,t:PointerTool,i:Int,h:Int):Float=if(t==PointerTool.FINGER||t==PointerTool.UNKNOWN)0f else if(h>=0)e.getHistoricalAxisValue(MotionEvent.AXIS_ORIENTATION,i,h) else e.getAxisValue(MotionEvent.AXIS_ORIENTATION,i)
}
```

```kotlin
package com.wetinknext.ui.theme

import androidx.compose.ui.graphics.Color

data class WetInkNextPalette(
    val appBg: Color,
    val panelBg: Color,
    val panelStroke: Color,
    val textPrimary: Color,
    val textSecondary: Color,
    val accent: Color,
) {
    companion object {
        val default = WetInkNextPalette(
            appBg = Color(0xFF17181C),
            panelBg = Color(0xFF25272D),
            panelStroke = Color(0xFF3A3D46),
            textPrimary = Color(0xFFF3F4F6),
            textSecondary = Color(0xFFA4A8B3),
            accent = Color(0xFF4D7AFF),
        )
    }
}
```

app/src/test/java/com/wetinknext/engine/brush/ColorSpacesTest.kt

```
package com.wetinknext.engine.brush
import org.junit.Assert.*
import org.junit.Test
class ColorSpacesTest { @Test fun blackAndWhiteAreStable(){val b=FloatArray(3);val w=FloatArray(3);ColorSpaces.srgb8ToLinear(0xF
F000000L,b);ColorSpaces.srgb8ToLinear(0xFFFFFFFFL,w);assertEquals(0f,b[0],.0001f);assertEquals(1f,w[0],.0001f)} @Test fun roundT
rip(){assertEquals(.25f,ColorSpaces.srgbToLinear(ColorSpaces.linearToSrgb(.25f)),.0001f)} }
```

```
1  package com.wetinknext.engine.brush
2
3  import org.junit.Assert.*
4  import org.junit.Test
5
6  class DabBufferTest {
7      @Test fun overflowIsCounted(){val b=DabBuffer(2);assertTrue(b.add(0f,0f,1f,0f,1f));assertTrue(b.add(1f,1f,1f,0f,1f));assertF
   alse(b.add(2f,2f,1f,0f,1f));assertEquals(1L,b.overflowCount)}
8      @Test fun clearResets(){val b=DabBuffer(1);b.add(0f,0f,1f,0f,1f);b.clear();assertEquals(0,b.count)}
9  }
```

```
package com.wetinknext.engine.brush

import org.junit.Assert.assertEquals
import org.junit.Assert.assertFalse
import org.junit.Test

class OneEuroFilterTest {
    @Test fun firstSamplePassesThrough() { val f=OneEuroFilter();val out=FloatArray(2);f.filter(0,10f,20f,out);assertEquals(10f,out[0],.0001f);assertEquals(20f,out[1],.0001f) }
    @Test fun samplesRemainFinite() { val f=OneEuroFilter();val out=FloatArray(2);f.filter(0,0f,0f,out);f.filter(8_000_000,10f,5f,out);assertFalse(out[0].isNaN());assertFalse(out[1].isInfinite()) }
}
```

```
package com.wetinknext.engine.brush

import org.junit.Assert.assertEquals
import org.junit.Assert.assertTrue
import org.junit.Test

class StabilizerTest {
    @Test fun strengthZeroPassesRawThrough(){val s=Stabilizer();s.process(0,100f,200f);assertEquals(100f,s.x,.0001f);assertEquals(200f,s.y,.0001f)}
    @Test fun velocityIsPositiveAfterMovement(){val s=Stabilizer();s.process(0,0f,0f);s.process(1_000_000_000,100f,0f);assertTrue(s.velocity>0f)}
}
```

```
package com.wetinknext.engine.brush

import com.wetinknext.engine.input.*
import org.junit.Assert.assertEquals
import org.junit.Test

class StampEmitterTest {
    @Test fun constantSpacingWithoutEndpointDuplicate(){val e=StampEmitter(BrushSettings(baseRadiusPx=10f,spacing=10f,spacingUsesDiameter=false,pressureToSize=false,pressureToOpacity=false));val d=DabBuffer(16);val down=InputBatch(1).apply{begin(InputAction.DOWN);addSample(0f,0f,1f,0f,0f,0f,0,0,PointerTool.STYLUS,false)};val move=InputBatch(1).apply{begin(InputAction.MOVE);addSample(20f,0f,1f,0f,0f,0f,1,0,PointerTool.STYLUS,false)};e.begin(down,d);e.append(move,d);e.finish(d,false);assertEquals(3,d.count)}
}
```

app/src/test/java/com/wetinknext/engine/core/DirtyRectTest.kt

```
package com.wetinknext.engine.core

import org.junit.Assert.assertArrayEquals
import org.junit.Assert.assertTrue
import org.junit.Test

class DirtyRectTest {
    @Test fun includesDabsAndConvertsToPixelBounds() {
        val rect = DirtyRect()
        rect.include(100f, 120f, 10f)
        rect.include(150f, 140f, 5f)
        val pixels = IntArray(4)
        rect.toPixelBounds(pixels)
        assertArrayEquals(intArrayOf(90, 110, 155, 145), pixels)
    }

    @Test fun clampCanMakeRectEmpty() {
        val rect = DirtyRect()
        rect.include(-100f, -100f, -10f)
        rect.clamp(100f, 100f)
        assertTrue(rect.isEmpty)
    }

    @Test fun clearResetsRect() {
        val rect = DirtyRect()
        rect.include(0f, 0f, 10f)
        rect.clear()
        assertTrue(rect.isEmpty)
    }
}
```

app/src/test/java/com/wetinknext/engine/core/ViewTransformFboTest.kt

```
package com.wetinknext.engine.core
import org.junit.Assert.assertEquals
import org.junit.Test
class ViewTransformFboTest { @Test fun orthoValues(){val m=FloatArray(16);ViewTransform.buildCanvasToFbo(100f,200f,m);assertEqua
ls(.02f,m[0],.0001f);assertEquals(.01f,m[5],.0001f);assertEquals(-1f,m[12],.0001f);assertEquals(-1f,m[13],.0001f)} }
```

app/src/test/java/com/wetinknext/engine/core/ViewTransformTest.kt

```
package com.wetinknext.engine.core

import org.junit.Assert.assertEquals
import org.junit.Test

class ViewTransformTest {
    @Test fun canvasAndScreenCoordinatesAreInverse() {
        val transform = ViewTransform(120f, 80f, 2.5f, 0.35f, true)
        val screen = FloatArray(2)
        val canvas = FloatArray(2)
        transform.canvasToScreen(240f, 310f, screen)
        transform.screenToCanvas(screen[0], screen[1], canvas)
        assertEquals(240f, canvas[0], 0.001f)
        assertEquals(310f, canvas[1], 0.001f)
    }

    @Test fun zoomAroundAnchorKeepsAnchorStable() {
        val initial = ViewTransform(translateX = 20f, translateY = 30f)
        val before = FloatArray(2)
        val after = FloatArray(2)
        initial.screenToCanvas(400f, 300f, before)
        initial.zoomAround(400f, 300f, 3f).screenToCanvas(400f, 300f, after)
        assertEquals(before[0], after[0], 0.001f)
        assertEquals(before[1], after[1], 0.001f)
    }
}
```